```
} catch(int) {
cout << "inside handler" << endl;
}
} ///:~
```

The output of this program is:

```
UseResources()
Rat()
Rat()
Rat()
allocating a Frog
inside handler
```

Here we have entered the `UseResources` constructor. The `Rat` constructor is also completed successfully for the array objects. We can see that an exception `Frog::operator` is also used. The problem here is that it ends inside the handler without calling the `UseResources` destructor. The `UseResources` could not be finished. It indicates that the `Cat` object was created successfully and was not destroyed.

7.3 Types of Constructors

There are various types of constructors - default constructors, copy constructors and dynamic constructors.

A Default Constructor is a special category of constructor that does not accept any parameter. For example, we can say that if `marketing` is a class, then `marketing::marketing()` is a default constructor because it doesn't need any parameter. The compiler will provide the default constructor. Constructors are not particularly defined in a class. It must not have any argument. The default constructor that is provided by the compiler does not have any special activities. What it can do is initialise data members that include a dummy value.